Reference: Engine API

This page describes every public function and data structure in `engine/`. It states facts only. For the reasons behind the design, read `docs/explanation/tiers-explained.md`. For results and figures, read `docs/white-paper.md`.

Version: 2.1.0 (2026-10-13).

Scoring rule (all Tier 2 and Daily Double functions)

A tie for first goes to a tiebreaker clue. This rule applies to every episode since 24 November 2014. The engines return a **win share**:

Final result	Win share
Outright first place	1
k-way tie for first place	1/k
Any other place	0
Top score is $0 or less	0 for every player

Tier 1 (`fj_wager.py`) does not use this rule. Tier 1 requires an outright win.

`engine/behavior.py` — simulated contestant behavior

This module holds every number that describes contestant behavior and the scoring rule. All values are in one configuration block at the top of the file. `app/index.html` mirrors the block in its `BEHAVIOR` object. The sources and the reasons are in `docs/adr/0003-simulated-contestant-behavior.md` and `docs/adr/0004-modern-era-calibration.md`.

Configuration constants

Each value has one of three labels:

- **CALIBRATED**: fitted privately to modern-era play (Seasons 38-41, 2021-2025, ADR-0004). The source data must not be republished, so this page does not give these values. See `engine/behavior.py` for the value.
- **PUBLISHED**: from Tesauro et al. (2013), which describes games from 1990s-2012.
- **ASSUMED**: our choice.

Name	Status	Meaning
`TIEBREAKER_RULE`	Rule since 2014-11-24	`True` (current value): a k-way tie scores 1/k. `False`: a tie scores 1.
`FJ_ACCURACY_BY_RANK`	CALIBRATED	Tuple of three Final Jeopardy accuracies, by score rank on entry to Final Jeopardy: leader, second, third. See `engine/behavior.py` for the value.
`FJ_ACCURACY_OVERALL`	CALIBRATED	Final Jeopardy accuracy over all players. Default for the analyzed player inside Daily Double rollouts, and for `OpponentModel.p_correct`. See `engine/behavior.py` for the value.

Name	Status	Meaning
`FJ_OUTCOME_CORRELATION`	CALIBRATED	Pairwise correlation of right/wrong outcomes across players, read as a binary correlation. `latent_correlation()` converts it. See `engine/behavior.py` for the value.
`DD_ACCURACY`	CALIBRATED	Daily Double accuracy. Default for future Daily Doubles in a rollout. See `engine/behavior.py` for the value.
`DD_WAGER_FRACTION_BY_ROUND`	CALIBRATED	Dict `{round: fraction}`. The fraction of the finder's score that a future Daily Double wager takes, for round 1 and round 2. See `engine/behavior.py` for the value.
`DD_FINDER_WEIGHTS_BY_RANK`	CALIBRATED	Tuple of three weights: the relative chance that the leader, second or third finds a future Daily Double. See `engine/behavior.py` for the value.
`B_WEIGHTS`	PUBLISHED for one regime (1990s-2012), ASSUMED for five	Second-place wager mix: `bankroll`, `keepout_c`, `overtake_a`, `two_thirds`, `random`. No modern calibration exists.
`A_WEIGHTS`	Cover share CALIBRATED; split ASSUMED	Leader wager mix when not locked out: `cover`, `cover_tie`, `bankroll`, `random`. See `engine/behavior.py` for the value.
`A_LOCKOUT_WEIGHTS`	ASSUMED	Leader wager mix in a lockout: `zero` and `max_safe`, equal weights.
`RANDOM_GRID_QUARTERS`	ASSUMED	`(0, 1, 2, 3, 4)`. The "random" bet: an equal spread over 0, 1/4, 1/2, 3/4 and all of the score.
`QUAD_HALF_WIDTH`	Numerical	`6.0`. Integration range [-6, 6] for the shared clue-difficulty factor.
`QUAD_STEP`	Numerical	`0.25`. Integration step for the shared factor.

`WagerDist` is a type alias: a list of `(wager, probability)` pairs.

`win_share(final_scores, player_index) -> float`

Returns the win share of one player for a set of final scores. See "Scoring rule" above.

`wager_distribution(scores, player_index) -> WagerDist`

Returns the simulated Final Jeopardy wager distribution for one player. The function gives each player a role by score: leader (A), second (B), third (C). Equal scores go to the lower index first.

- A player with a score of $0 or less wagers $0.
- A, not locked out: uses `A_WEIGHTS`.
- A, in a lockout (A > 2B): uses `A_LOCKOUT_WEIGHTS`.
- B: uses `B_WEIGHTS`.
- C: uses the random grid only.

If a bet type does not exist in a state, the function shares its weight among the other bet types, in proportion. Each wager is clamped to [0, score]. The probabilities sum to 1.

`score_rank(scores, player_index) -> int`

Returns the score rank of one player: 0 for the leader, 1 for second, 2 for third. Equal scores go to the lower index first.

`fj_accuracy(scores, player_index) -> float`

Returns the Final Jeopardy accuracy of a simulated contestant: `FJ_ACCURACY_BY_RANK[score_rank(scores, player_index)]`. A rank above 2 uses the last entry.

`sample_dd_finder(scores, rng) -> int`

Draws the index of the player who finds a future Daily Double. The draw uses `DD_FINDER_WEIGHTS_BY_RANK`, applied by score rank. `rng` is a `random.Random`.

`latent_correlation(binary_correlation) -> float`

Returns `sin(binary_correlation * pi / 2)`. This is the latent normal correlation that gives the stated binary correlation when every accuracy is p = 0.5 (Sheppard's formula). At other accuracies, the conversion is approximate (ADR-0004).

`normal_cdf(x) -> float` and `inverse_normal_cdf(p) -> float`

The standard normal CDF and its inverse. `inverse_normal_cdf` uses Acklam's approximation and one Halley step. The app uses the same method.

`quadrature_nodes() -> List[Tuple[float, float]]`

Returns `(factor, weight)` pairs for the shared clue-difficulty factor. The weights sum to 1.

`conditional_accuracy(p, factor, r) -> float`

Returns the probability of a correct answer, given the shared factor, in a one-factor Gaussian copula with latent correlation `r`. If `r <= 0`, it returns `p`.

`sample_correct(ps, r, rng) -> List[bool]`

Draws one correlated right/wrong outcome per player. `ps` is the list of accuracies. `r` is the latent correlation.

`sample_wager(dist, rng) -> int`

Draws one wager from a `WagerDist`.

`engine/fj_wager.py` — Tier 1 (guarantee-based)

`legal_max_wager(score: int) -> int`

Returns the largest legal Final Jeopardy wager: the score, or $0 if the score is $0 or less.

`worst_case_ceiling(score: int) -> int`

Returns the highest score an opponent can reach: the score, doubled. This assumes that the opponent wagers everything and answers correctly. A score of $0 or less is returned unchanged.

`analyze(scores: List[int], player_index: int) -> FJRecommendation`

Computes the Tier 1 wager for one player against all others.

Parameters:

Name	Type	Meaning
`scores`	list of int	The current score of each player, in any order.
`player_index`	int	The position, in `scores`, of the player to analyze.

Returns: an `FJRecommendation` with these fields:

Field	Type	Meaning
`player_index`	int	The analyzed player.
`player_score`	int	The score of the analyzed player.
`opponent_scores`	list of int	The scores of the other players.
`lockout`	bool	`True` if no opponent can catch this player, even with an all-in correct bet.
`binding_opponent_index`	int or None	The opponent whose worst case sets the requirement. This is the single largest threat, not a combination of opponents.
`min_wager_to_guarantee_win`	int or None	The smallest wager that beats the worst case of every opponent outright. `None` if no wager can guarantee a win.
`max_legal_wager`	int	The largest legal wager (the score of the player).
`lock_tie_wager`	int or None	A wager that locks in at least a tie, when one exists.
`notes`	list of str	Plain-language notes about this result.

`analyze_all(scores: List[int]) -> List[FJRecommendation]`

Runs `analyze()` for each player. Returns one `FJRecommendation` per player, in the order of `scores`.

`engine/fj_bestresponse.py` — Tier 2 (confidence-weighted)

Tier 2 is exact in double precision. It integrates over the shared clue-difficulty factor and enumerates every wager and every right/wrong branch. It does not sample.

Module constants

Name	Value
`DEFAULT_CORRELATION`	`behavior.FJ_OUTCOME_CORRELATION` (CALIBRATED)

`OpponentModel`

A data structure for one opponent.

Field	Type	Meaning
`score`	int	The current score of the opponent.
`wager_dist`	`WagerDist`	Each possible wager, with its probability. The probabilities must sum to 1.0.
`p_correct`	float	The probability that the opponent answers correctly. Default: `behavior.FJ_ACCURACY_OVERALL`. `analyze()` sets it for each opponent.

`default_opponent_wager_model(scores, opponent_index) -> WagerDist`

Returns `behavior.wager_distribution(scores, opponent_index)`: the simulated contestant wager distribution for that opponent.

`expected_win_probability(self_score, self_wager, p_self_correct, opponents, correlation=0.0) -> float`

Returns the exact expected win share for one wager, against a list of `OpponentModel` objects.

Name	Type	Meaning
`self_score`	int	The score of the analyzed player.
`self_wager`	int	The wager to evaluate.
`p_self_correct`	float	The confidence of the analyzed player, from 0.0 to 1.0.
`opponents`	list of `OpponentModel`	The opponents.
`correlation`	float	Binary correlation of right/wrong outcomes across all players, including the analyzed player. Default: **0.0** (independent outcomes). The default is 0 so that hand-derived tests can use independent outcomes. `analyze()` uses `DEFAULT_CORRELATION`.

`analyze(scores, player_index, p_self_correct, opponent_model_fn=default_opponent_wager_model, opponent_correctness=None, correlation=DEFAULT_CORRELATION, wager_step=100) -> FJBestResponse`

Tests candidate wagers. Returns the wager with the highest expected win share.

Parameters:

Name	Type	Meaning
`scores`	list of int	The current score of each player.
`player_index`	int	The position of the player to analyze. Raises `ValueError` if out of range.
`p_self_correct`	float	The confidence of the analyzed player, from 0.0 to 1.0.
`opponent_model_fn`	function	Makes the wager distribution of each opponent. Default: `default_opponent_wager_model`.
`opponent_correctness`	float or None	`None` (default): each opponent gets the accuracy for their score rank, `behavior.fj_accuracy(scores, i)`. A float: every opponent gets that accuracy.
`correlation`	float	Binary correlation of right/wrong outcomes across all players. Default: `DEFAULT_CORRELATION`.
`wager_step`	int	The gap, in dollars, between candidate wagers. Default: 100.

The candidate wagers are every multiple of `wager_step` from $0 to the legal maximum, plus the legal maximum, the Tier 1 cover wager, the cover wager minus $1, and the Tier 1 lock-tie wager, when these exist.

The candidates also include every breakpoint. For own score `S` and each final score `f` that an opponent can reach, the breakpoints are `f - S`, `f - S + 1`, `S - f` and `S - f - 1`. The win share changes only at these wagers. Thus the engine tries the best wager in each flat stretch exactly. The engine keeps only candidates from $0 to the legal maximum.

Choice rule: win share first, money second.

1. Find the highest expected win share.
2. Keep each candidate whose win share is within 1e-9 of it.
3. From these, choose the candidate with the highest expected winnings, rounded to 6 decimal places. Expected winnings are the final score times the win share, summed over all outcomes. The winner keeps their final total.
4. If two candidates have equal expected winnings, choose the smaller wager.

Returns: an `FJBestResponse` with these fields:

Field	Type	Meaning
`player_index`	int	The analyzed player.
`recommended_wager`	int	The wager with the highest expected win share. If two or more wagers are within 1e-9 of the best, the wager with the highest expected winnings is chosen, then the smallest (see "Choice rule").
`win_probability`	float	The expected win share at the recommended wager. A tie scores 1/k (see "Scoring rule").
`wager_win_probabilities`	list of (int, float)	Each candidate wager, with its expected win share.
`tier1_cover_wager`	int or None	The Tier 1 guarantee wager, for comparison.
`notes`	list of str	The tie rule, the opponent model, and the accuracy and correlation used.

`engine/simulate.py` — rollout model for the Daily Double engine

Module constants

Name	Value	Status
`DEFAULT_REGULAR_CLUE_SUCCESS_RATE`	0.6	ASSUMED (placeholder)
`DEFAULT_FJ_CONFIDENCE`	`behavior.FJ_ACCURACY_OVERALL`	CALIBRATED
`DEFAULT_FUTURE_DD_SUCCESS_RATE`	`behavior.DD_ACCURACY`	CALIBRATED

`RolloutConfig`

The assumptions for one simulated rest of the game, after a Daily Double.

Field	Type	Default	Meaning
`remaining_clue_values`	list of int	`[]`	The dollar value of each regular clue still on the board.
`remaining_dd_count`	int	0	The number of Daily Doubles still hidden.

Field	Type	Default	Meaning
`regular_clue_success_rate`	float	0.6	The chance that a player answers a given regular clue correctly.
`fj_confidence`	float	`DEFAULT_FJ_CONFIDENCE`	The Final Jeopardy accuracy of the analyzed player inside the rollout.
`opponent_fj_accuracy`	float or None	`None`	`None`: each opponent gets the accuracy for their score rank (`behavior.fj_accuracy`). A float: every opponent gets that accuracy.
`fj_correlation`	float	`behavior.FJ_OUTCOME_CORRELATION`	Binary correlation of Final Jeopardy outcomes across all players.
`future_dd_wager_fraction`	float or None	`None`	`None`: the calibrated fraction for `round_number` (`behavior.DD_WAGER_FRACTION_BY_ROUND`). A float: that fraction of the finder's score.
`future_dd_success_rate`	float	`DEFAULT_FUTURE_DD_SUCCESS_RATE`	The chance that the finder answers a future Daily Double correctly.
`round_number`	int	2	The round of the future Daily Doubles. `dd_wager.analyze()` sets it to the round of the current Daily Double.

`simulate_remaining_regular_clues(scores, clue_values, success_rate=0.6, rng=None) -> List[int]`

For each clue value, a uniformly random player gains the value with probability `success_rate`. Returns the new scores.

`simulate_future_daily_doubles(scores, count, wager_fraction=None, success_rate=DEFAULT_FUTURE_DD_SUCCESS_RATE, rng=None, round_number=2) -> List[int]`

Plays `count` future Daily Doubles. For each one, `behavior.sample_dd_finder` picks the finder by score rank. The finder wagers `wager_fraction` of their score (not less than $0) and wins or loses it. `wager_fraction=None` uses `behavior.DD_WAGER_FRACTION_BY_ROUND[round_number]`; an unknown round uses the round 2 value. Returns the new scores.

`tier1_wager(scores, i) -> int`

Returns the Final Jeopardy wager of the analyzed player inside a rollout: $0 in a lockout, else the Tier 1 lock-tie wager, else the Tier 1 cover wager, else the legal maximum.

`simulate_final_jeopardy(scores, player_index, fj_confidence=DEFAULT_FJ_CONFIDENCE, opponent_accuracy=None, correlation=behavior.FJ_OUTCOME_CORRELATION, rng=None) -> List[int]`

Resolves Final Jeopardy once. The analyzed player (`player_index`) uses `tier1_wager`. Each opponent draws a wager from `behavior.wager_distribution`. `opponent_accuracy=None` gives each opponent the accuracy for their score rank (`behavior.fj_accuracy`); a float gives every opponent that accuracy. Right/wrong outcomes are correlated across all players. Returns the final scores.

`wins(scores, player_index) -> float`

Returns `behavior.win_share(scores, player_index)`: 1, 1/k, or 0.

`rollout(scores, player_index, config, rng=None) -> float`

Simulates the rest of the game once. Each future Daily Double falls at a uniformly random point among the remaining regular clues. The regular clues and the future Daily Doubles play in that order, then Final Jeopardy. Returns the win share of `player_index` (1, 1/k, or 0).

`engine/dd_wager.py` — Daily Double engine

`legal_max_dd_wager(score: int, round_number: int) -> int`

Returns the largest legal Daily Double wager: the greater of the score and the round cap ($1,000 in Single Jeopardy, $2,000 in Double Jeopardy).

`analyze(scores, player_index, round_number, p_correct, config=None, wager_step=100, trials_per_wager=3000, z_score=None, alpha=0.05, seed=None) -> DDRecommendation`

Runs a Monte Carlo simulation of the rest of the game for each candidate wager. All candidates use the same random numbers in each trial (common random numbers). Returns the smallest wager that the simulation cannot separate from the best one, except for exact ties (see `recommended_wager`).

Parameters:

Name	Type	Meaning
`scores`	list of int	The current score of each player.
`player_index`	int	The position of the player to analyze. Raises `ValueError` if out of range.
`round_number`	int	`1` for Single Jeopardy, `2` for Double Jeopardy.
`p_correct`	float	The confidence of the analyzed player in this Daily Double clue.
`config`	`RolloutConfig` or None	The rest of the game. `None` gives `RolloutConfig()`. The engine uses a copy of `config` with `round_number` set to this round, so future Daily Doubles use the wager fraction of this round. The caller's object does not change.
`wager_step`	int	The gap, in dollars, between candidate wagers. Default: 100.
`trials_per_wager`	int	Simulated trials per candidate wager. Default: 3000.
`z_score`	float or None	A fixed band width for the plateau. `None` (default) uses a Bonferroni band, `z* = Phi^-1(1 - alpha/k)`, for `k` candidate wagers.
`alpha`	float	The family-wise error level for the Bonferroni band. Default: 0.05. The engine ignores it when `z_score` is set.
`seed`	int or None	A fixed random seed. The same seed gives the same result.

Returns: a `DDRecommendation` with these fields:

Field	Type	Meaning
`player_index`	int	The analyzed player.
`player_score`	int	The score of the analyzed player.
`round_number`	int	The round.

Field	Type	Meaning
`p_correct`	float	The confidence used.
`recommended_wager`	int	The smallest wager on the plateau. Exception: if `p_correct > 0.5`, the largest wager whose per-trial win share equals that of the smallest plateau wager on every trial (an exact tie, for example in a solo game).
`win_probability`	float	The mean win share at the best candidate. This is the in-sample maximum, so it is slightly optimistic (RISK_REGISTER.md risk #11).
`wager_win_probabilities`	list of (int, float)	Each candidate wager, with its mean win share.
`standard_errors`	list of (int, float)	Each candidate's standard error, `sqrt(p(1-p)/trials)`.
`comparison_standard_errors`	list of (int, float)	Each candidate's paired standard error against the best candidate.
`plateau`	list of int	Each wager that the paired comparison cannot separate from the best one.
`z_score_used`	float	The band width used.
`trials_per_wager`	int	The trial count used.
`notes`	list of str	Plain-language caveats for this result.